

Coming from superpowers?

You already know 80% of chris-code.

Same `brainstorm → plan → execute → review → finish` pipeline.
Same skill names. Same TDD and systematic-debugging discipline.

This deck is the other **20%**, and the *why*.

What feels different, day to day

1

Shorter specs & plans

No more 2,500-word specs or 10k-word plans full of code the implementer throws away.

2

Dedicated agents

Coding and review run through named agents auto-dispatched by file type. You rarely pick one.

3

Same gates, every task

Spec compliance → code quality → pre-commit lint. Nothing is "too small to review."

Five thematic shifts

- 1 **Lean artifacts** over exhaustive ones
- 2 **A real agent layer**, dispatched by scope
- 3 **Review** is a uniform, multi-stage gate
- 4 **Parallelism** is a feature, not a footgun
- 5 **Native tools** first, with a hard gate

1 • Lean artifacts over exhaustive ones

superpowers

Spec written "as if zero context"

Plan: complete code in every step

specs grew past 2,500 w • plans 5,000–10,000 w

agents discarded the pasted code and rewrote it



chris-code

Spec = invariants only

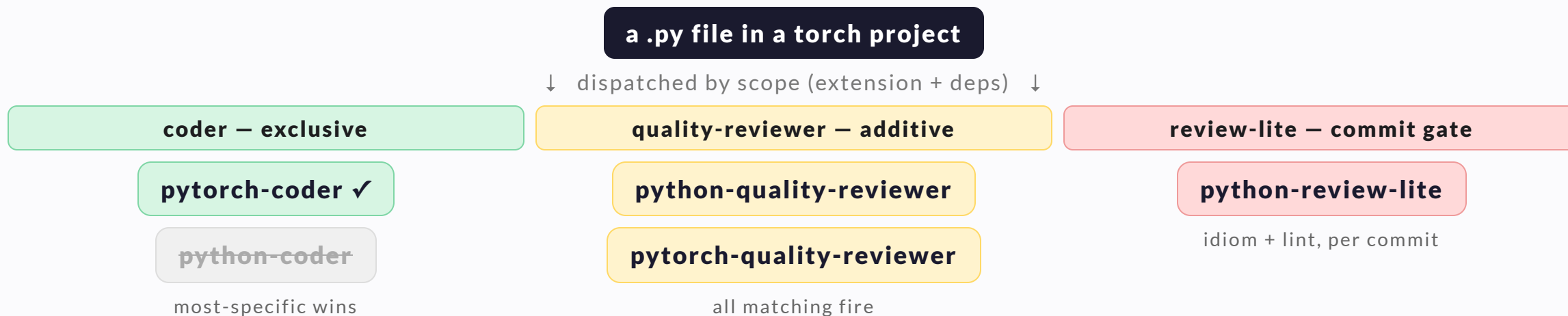
Plan = what & where, no code

code written against the real repo

"contracts stay, choreography goes"

2 • A real agent layer, dispatched by scope

superpowers dispatches a generic `Task (general-purpose)` subagent steered by a prompt template. chris-code routes to dedicated agents:



3 • Review: kept the spine, hardened it



Same two-stage spine. chris-code makes the reviewers **dedicated, scope-dispatched agents**, adds a **mandatory lint gate**, and re-reads the actual code at every gate.

4 • Parallelism is a feature, not a footgun

✗ superpowers — "dispatch parallel implementers" is a Red Flag

implementer A

implementer B

implementer C

unscheduled: they edit the same files and collide

✓ chris-code — map file footprints, then stage

Stage 1

A • api/

B • db/

C • ui/

Stage 2

D • api/ + db/ (waits — overlaps A & B)

5 • Native tools first, with a hard gate



The split is only at "no native tool": superpowers falls back; chris-code refuses, because a silent fallback once caused real damage.

What stayed the same

- ✓ **brainstorming** — intent → approaches → design
- ✓ **test-driven-development** — RED-GREEN-REFACTOR
- ✓ **systematic-debugging** — four phases, Iron Law
- ✓ **finishing-a-development-branch** — merge / PR / keep / discard
- ✓ **dispatching-parallel-agents, receiving-code-review, using-chris-code** (light edits)

Skills you know – what changed most

Where muscle memory will mislead you. Framed before → after:

Skill	superpowers	chris-code
writing-plans	the plan skill: exhaustive, full code per step	slimmed to <code>lean-plan</code> ; spec → <code>lean-spec</code>
subagent-driven-development	2-stage review; parallel discouraged	3 gates/task, scope dispatch, staged parallelism
verification-before-completion	"what command proves this? run it"	4 steps: tests → lints → review → requirements
requesting-code-review	primary, mandatory path	demoted to ad-hoc; base <code>HEAD~1</code> → <code>merge-base HEAD main</code>

What's new, and why

New skills (9)



New agents (9) — the layer superpowers doesn't have

3 coders (`python` / `pytorch` / `rust`) — exclusive, most-specific wins

3 quality-reviewers — additive post-spec review

2 review-lite gates — pre-commit idiom + lint

`bug-hunter` — adversarial edge-case test writer

The net

chris-code is a **true superset**: every superpowers skill is present (one renamed, one split), plus 9 new skills and a 9-agent layer.

Same spine. Leaner artifacts, mechanical dispatch, harder gates.

Full write-up: <docs/superpowers-comparison.md>